

Will hardly anyone bid? — TenderMining

The idea: the moment a German construction tender is published, predict whether it will end with **0 or 1 bids**. For a contractor such a "lonely" tender is the best business there is — no price war, a near-certain win. Officially you only learn the bid count months later, when the award is published. This model predicts it on day one, from nothing but the public notice.

Background: all German public construction tenders and their outcomes are published as open data (TED, the EU's tender journal). Outcomes arrive months after the call for bids — so a machine that reads today's notices and has learned from thousands of past outcomes can see patterns no single bidder can.

The headline result, measured on 3 months of tenders the model had never seen (Apr–Jun 2026):

out of 100 tenders, how many end with 0–1 bids?	
picking tenders at random ("chance")	17
picking only tenders the model flags	37

How this notebook is organized:

1. **The data** — what a tender notice contains, how outcomes were attached, and how hindsight was strictly kept out.
2. **Train & test honestly** — the model must beat random picking *and* a one-trick "trade code only" model, on months it never saw.
3. **The findings** — the score is a trustworthy *ordering* but not a quotable *percentage*; and the sorting works even inside a single trade, shown on a gold-mine business and a cut-throat business.
4. **Trust checks** — four tripwires that would catch the model cheating (peeking at the future). All four pass.
5. **What to do next** — the product, and who to talk to first.

Every number in this notebook is printed by a code cell — nothing is quoted from outside.

How the model thinks — in plain words

The model is a large collection of **decision trees**. A single tree is just a chain of simple yes/no questions about the notice: "*Is the project longer than 6 months?*" → "*Is a bid bond required?*" → "*Is it electrical work?*" — and at the end of the chain it gives a small nudge toward "few bidders" or "many bidders".

One tree alone is a poor guesser. So training builds trees one after another, and **each new tree focuses on correcting the mistakes of the previous ones** (this technique is called *gradient boosting*; CatBoost, the library used below, is one implementation of these *gradient-boosted trees*). The final risk score simply adds up thousands of these small votes.

The important consequence: **no single question decides anything**. The prediction is many weak clues combined — like an experienced estimator's gut feeling ("public bath renovation, short deadline, bid bond required, middle of nowhere... nobody will bid on this"), except written down, measurable, and testable.

Part 1 — The data

First, install the machine-learning library (**CatBoost**, which builds the gradient-boosted decision trees described above — a standard choice for table-shaped data of this size) and the usual Python tools.

```
In [ ]: # Setup: CatBoost + imports (CPU runtime is intentional - ~4k rows)
%pip install -q catboost
import numpy as np
import pandas as pd
import pyarrow.parquet as pq
from catboost import CatBoostClassifier
from sklearn.metrics import average_precision_score, roc_auc_score
import catboost, sklearn
print('catboost', catboost.__version__, '| pandas', pd.__version__, '| sklearn',
      sklearn.__version__)
```

catboost 1.2.10 | pandas 2.2.2 | sklearn 1.6.1

Load the data from Google Drive. Two files, stored as **parquet** (a compact file format for tables): **tenders** = what each notice said on the day it was published (our inputs — ~18,800 notice versions), and **awards** = how each lot eventually ended, including the bid count (the answer sheet — ~5,400 decided lots).

```
In [ ]: # Mount Google Drive and locate the two parquet extracts
from google.colab import drive
drive.mount('/content/drive')

import glob
TENDERS_NAME = 'tenders_cpv45_20260101-20260630.parquet'
AWARDS_NAME = 'awards_cpv45_20260101-20260630.parquet'
tenders_hits = glob.glob(f'/content/drive/MyDrive/**/{TENDERS_NAME}', recursive=True)
awards_hits = glob.glob(f'/content/drive/MyDrive/**/{AWARDS_NAME}', recursive=True)
print('tenders:', tenders_hits)
print('awards:', awards_hits)
assert len(tenders_hits) >= 1 and len(awards_hits) >= 1, 'parquet files not found in Drive'
TENDERS_PATH, AWARDS_PATH = tenders_hits[0], awards_hits[0]
```

Mounted at /content/drive

tenders: ['/content/drive/MyDrive/tenders_cpv45_20260101-20260630.parquet']

awards: ['/content/drive/MyDrive/awards_cpv45_20260101-20260630.parquet']

```
In [ ]: # Load parquets; read the `role` tag embedded in each column's parquet metadata (FIELDS.md)
def load_with_roles(path):
    schema = pq.read_schema(path)
    roles = {}
    for name in schema.names:
        md = schema.field(name).metadata or {}
        roles[name] = md.get(b'role', b'').decode() or None
    df = pd.read_parquet(path)
    return df, roles

tenders, tender_roles = load_with_roles(TENDERS_PATH)
awards, award_roles = load_with_roles(AWARDS_PATH)

print('tenders rows:', len(tenders), '| cols:', tenders.shape[1])
print('awards rows:', len(awards), '| cols:', awards.shape[1])
print('tenders role counts:', pd.Series([r or 'MISSING' for r in tender_roles.values()]).value_counts().to_dict())
missing_roles = [c for c, r in tender_roles.items() if r is None]
print('columns without role metadata:', missing_roles)
# revision structure
key = ['procedure_id', 'lot_id']
print('unique tender lots:', tenders.groupby(key).ngroups)
print('revision count distribution:', tenders.groupby(key).size().value_counts().sort_index().to_dict())
print('unique award lots:', awards.groupby(key).ngroups)
```

tenders rows: 18813 | cols: 103

awards rows: 5387 | cols: 46

tenders role counts: {'categorical': 28, 'numeric': 20, 'bool': 14, 'plumbing': 9, 'date': 7, 'key': 6, 'hierarchical': 6, 'text': 5, 'nested': 4, 'entity': 4}

columns without role metadata: []

unique tender lots: 15414

revision count distribution: {1: 12970, 2: 1824, 3: 381, 4: 176, 5: 45, 6: 9, 7: 6, 8: 1, 9: 1, 10: 1}

unique award lots: 5356

Build the answer sheet. A lot counts as "low competition" if it ended with **0 or 1 bids**.

That outcome is attached to every published version of the lot's notice. The firewall rule: the model's inputs come **only** from what was public on publication day — every other column from the awards file is dropped immediately, and an assertion (an automatic check that halts the notebook with an error) fails if any slips through. Result: ~5,000 notice versions covering ~3,900 decided lots; about 1 in 10 ended with 0–1 bids.

```
In [ ]: # Dataset assembly (TRAINING.md Leakage rule 1: source firewall)
# 1) Latest award revision per lot supplies the label
aw = awards.sort_values('publication_date').groupby(key,
as_index=False).tail(1).copy()

# 2) Drop reporting errors before anything else
def has_flag(flags, name):
    if flags is None: return False
    try: return name in list(flags)
    except TypeError: return False
bad = aw['quality_flags'].apply(lambda f: has_flag(f, 'winner_but_zero_tenders'))
print(f'dropping {bad.sum()} awards flagged winner_but_zero_tenders')
aw = aw[~bad]
aw = aw[aw['n_tenders'].notna()]

# 3) Label: insufficient competition = 0 or 1 bids
aw['label'] = (aw['n_tenders'] <= 1).astype(int)

# 4) Firewall: from awards keep ONLY join keys + label. Everything else is post-
outcome.
aw_label = aw[key + ['label']].copy()

# 5) Every tender revision is a row; inner join attaches the lot's eventual label
data = tenders.merge(aw_label, on=key, how='inner')
awards_cols_leaked = [c for c in data.columns if c in set(awards.columns) -
set(tenders.columns) - {'label'}]
assert awards_cols_leaked == [], f'awards columns leaked into features:
{awards_cols_leaked}'

n_lots = data.groupby(key).ngroups
print(f'labeled rows (revisions): {len(data)} | labeled lots: {n_lots}')
lot_label = data.groupby(key)['label'].first()
print(f'lot-level base rate: {lot_label.mean():.4f} ({lot_label.sum()} positive
lots)')
```

```
dropping 1 awards flagged winner_but_zero_tenders
labeled rows (revisions): 5016 | labeled lots: 3945
lot-level base rate: 0.1044 (412 positive lots)
```

Turn each notice into 82 facts. The model may look at: what is being bought (**CPV codes** — the *Common Procurement Vocabulary*, the EU's catalogue number for products and services; every code starting with 45 is construction), where (**NUTS regions** — the EU's standard numbering of European regions, from state down to district — and postal zones), who is buying (type of authority — municipality, state agency; never the buyer's *name*, which would let the model memorize), money (estimated value), timing (deadlines, project duration), and hurdles for bidders (bid bond, required certificates, exclusion grounds). Fields are selected *mechanically* by the role tag stored in the data files — nobody hand-picks convenient columns. Empty fields stay visibly empty ("no bid bond mentioned" is itself a clue).

```

In [ ]: # Feature engineering (leakage rule 2: mechanical selection by role; unknown roles
        # excluded)
        # Defined as a function so the production dry-run (tripwire 4) reuses the exact same
        # code.
        NA = '__NA__'

        def as_list(v):
            if v is None: return []
            if isinstance(v, (list, np.ndarray)): return list(v)
            if isinstance(v, float) and np.isnan(v): return []
            return [v]

        def join_codes(v):
            vals = sorted({str(x) for x in as_list(v)})
            return '|'.join(vals) if vals else NA

        def cat_str(v):
            if v is None or (isinstance(v, float) and np.isnan(v)): return NA
            return str(v)

        def hier_levels(col):
            if 'cpv' in col: return [('cpv2', 2), ('cpv3', 3), ('cpv4', 4)]
            if 'nuts' in col: return [('nuts1', 3), ('nuts2', 4), ('nuts3', 5)]
            if 'postal' in col: return [('zone1', 1)]
            return None

        # List-typed columns detected once on the full tenders frame, so any subset
        # transforms identically
        IS_LIST = {c: tenders[c].map(lambda v: isinstance(v, (list, np.ndarray))).any() for c
                    in tenders.columns}

        def build_features(df):
            Xf = pd.DataFrame(index=df.index)
            cats, nums, excl = [], [], []
            for col, role in tender_roles.items():
                s = df[col]
                if role == 'numeric':
                    Xf[col] = pd.to_numeric(s, errors='coerce'); nums.append(col)
                elif role == 'bool':
                    # 3-way categorical: missingness is informative (bid_bond_required null
                    != False)
                    Xf[col] = s.map(lambda v: NA if v is None or (isinstance(v, float) and
                    np.isnan(v)) else str(bool(v)))
                    cats.append(col)
                elif role == 'categorical':
                    Xf[col] = s.map(join_codes) if IS_LIST[col] else s.map(cat_str)
                    cats.append(col)
                elif role == 'hierarchical':
                    levels = hier_levels(col)
                    if levels is None:
                        excl.append((col, 'hierarchical-unknown-scheme')); continue
                    for lname, n in levels:
                        new = f'{col}__{lname}'
                        if IS_LIST[col]:
                            Xf[new] = s.map(lambda v, n=n: '|'.join(sorted({str(x)[:n] for x
                            in as_list(v)})) or NA)
                        else:
                            Xf[new] = s.map(lambda v, n=n: NA if v is None or (isinstance(v,
                            float) and np.isnan(v)) else str(v)[:n])
                            cats.append(new)
                elif role == 'date':
                    if col == 'publication_date': continue # the reference point, not a
                    feature
                    Xf[f'span__{col}'] = (pd.to_datetime(s) -

```

```

pd.to_datetime(df['publication_date'])).dt.days
    nums.append(f'span_{col}')
    else:
        excl.append((col, role or 'MISSING'))
    return Xf, cats, nums, excl

X, cat_cols, num_cols, excluded = build_features(data)
assert 'buyer_name' in [c for c, _ in excluded], 'buyer_name must be excluded (rule 4)'
FEATURES = cat_cols + num_cols
print(f'features: {len(FEATURES)} ({len(cat_cols)} categorical, {len(num_cols)} numeric)')
print('excluded roles:', pd.Series([r for _, r in excluded]).value_counts().to_dict())

```

features: 82 (56 categorical, 26 numeric)
excluded roles: {'plumbing': 9, 'key': 6, 'text': 5, 'nested': 4, 'entity': 4}

A guard against self-deception. There are two ways to feed a category (like a region code) to a model. As a plain yes/no fact ("is this lot in Bavaria?") — harmless; the technical name is **one-hot** encoding. Or by replacing it with its average outcome ("Bavaria: 13% single-bid") — dangerous, because the feature is then built out of the answers; that is called **target statistics**. CatBoost silently switches from the first to the second when a field has too many distinct values (its **cardinality**), so this cell counts the distinct values of every field and makes sure CatBoost always stays on the harmless one-hot path and can never switch to the dangerous one.

```

In [ ]: # Cardinality check for Leakage rule 4 (pure one-hot, no target statistics).
        # Spec says one_hot_max_size=128 assuming all categoricals fit under it; the
        # engineered
        # prefix/combo columns exceed that (selection_criteria_types=650,
        # cpv_additional__cpv4=438,
        # nuts3 levels ~300+). Above one_hot_max_size CatBoost silently switches to target
        # statistics – the exact thing rule 4 forbids – so we raise the knob to keep every
        # column one-hot. The rule's substance (no target statistics) is preserved.
        card = pd.Series({c: X[c].nunique() for c in cat_cols}).sort_values(ascending=False)
        print(card.head(10))
        ONE_HOT_MAX_SIZE = 1024
        assert card.max() <= ONE_HOT_MAX_SIZE, 'raise ONE_HOT_MAX_SIZE: a categorical exceeds
        it -> CatBoost would use target statistics'
        print(f'one_hot_max_size={ONE_HOT_MAX_SIZE} covers max cardinality {card.max()} ->
        all one-hot, no CTR')

```

selection_criteria_types	650
cpv_additional__cpv4	438
place_nuts3__nuts3	336
buyer_nuts__nuts3	300
cpv_additional__cpv3	210
exclusion_grounds	107
cpv_additional__cpv2	88
procurement_additional_types	68
platform_name	50
place_nuts3__nuts2	39

dtype: int64
one_hot_max_size=1024 covers max cardinality 650 -> all one-hot, no CTR

The honest exam setup. We pretend today is **23 March 2026**: the model learns only from lots first published *before* that date, and is graded only on lots published *after* it — never seeing them during training. (Splitting by date instead of at random is called a **temporal split**; it's the fair test, because in real life you always predict the future from the past.) The code asserts that no lot ends up on both sides. Corrected notices count once per version, but each version of a 5-version lot only carries 1/5 of a vote, so heavily-corrected lots don't dominate.

```
In [ ]: # Temporal group-aware split (leakage rule 3): a lot goes wholly to train or test
# by its FIRST publication_date; ~80/20; assert no lot straddles the boundary.
data['publication_date'] = pd.to_datetime(data['publication_date'])
first_pub = data.groupby(key)['publication_date'].transform('min')
lot_first = data.groupby(key)['publication_date'].min()
threshold = lot_first.quantile(0.8)
print('split threshold (80th pct of lot first publication):', threshold.date())

is_train = first_pub <= threshold
train_lots = set(map(tuple, data.loc[is_train, key].drop_duplicates().values))
test_lots = set(map(tuple, data.loc[~is_train, key].drop_duplicates().values))
assert not (train_lots & test_lots), 'a (procedure_id, lot_id) straddles the split
boundary'

# 1/k revision weighting, k = the lot's revision count (train AND eval)
k = data.groupby(key)['label'].transform('size')
data['weight'] = 1.0 / k

y = data['label'].values
w = data['weight'].values
Xtr, ytr, wtr = X[is_train.values], y[is_train.values], w[is_train.values]
Xte, yte, wte = X[~is_train.values], y[~is_train.values], w[~is_train.values]
print(f'train: {len(Xtr)} rows / {len(train_lots)} lots | test: {len(Xte)} rows /
{len(test_lots)} lots')
base_rate_test = np.average(yte, weights=wte)
print(f'weighted base rate - train: {np.average(ytr, weights=wtr):.4f} | test:
{base_rate_test:.4f}')
```

```
split threshold (80th pct of lot first publication): 2026-03-23
train: 4052 rows / 3159 lots | test: 964 rows / 786 lots
weighted base rate - train: 0.0890 | test: 0.1667
```

Part 2 — Train the model and compare it against chance

What "chance" means here: flag lots at random. Since ~17% of test lots ended with 0–1 bids, random flagging is right ~17% of the time — that's the number to beat, and it appears below as the "constant base" line (0.1667). This 17% is called the **base rate**: the share of single-bid lots you'd hit by pure guessing.

Second opponent: a one-trick model that only knows the trade code (CPV-4, e.g. "roofing works") and predicts each trade's historical rate. If our 82-fact model can't beat that, the notice details add nothing over just knowing the trade.

The two scores used below, spelled out once:

- **PR-AUC** = **P**recision–**R**ecall **A**rea **U**nder the **C**urve. *Precision* = of the lots we flag, the share that really ended single-bid. *Recall* = of all single-bid lots, the share we caught. Move the flagging cut-off and the two trade against each other (flag more → catch more, but more false alarms); plotted across every possible cut-off they form a curve, and the score is the area under it — one grade for the whole trade-off instead of one lucky cut-off. **0.17 (the base rate) = clueless, 1.00 = perfect.**
- **ROC-AUC** = **R**eceiver **O**perating **C**haracteristic **A**rea **U**nder the **C**urve — a name inherited from 1940s radar operators; ignore the words, they explain nothing. The useful translation: show the model one random single-bid lot and one random competitive lot — how often does it rank the single-bid one higher? **0.5 = coin flip, 1.0 = always right.** It is the standard score in the research literature, which is why the "too good to be true" check in Part 3 is phrased in it.


```

In [ ]: # Train CatBoost (TRAINING.md model block) and evaluate against both baselines
from catboost import Pool

def make_model():
    return CatBoostClassifier(
        cat_features=cat_cols,
        one_hot_max_size=ONE_HOT_MAX_SIZE, # pure one-hot, no target statistics
(rule 4)
        auto_class_weights='Balanced',
        eval_metric='PRAUC',
        random_seed=42,
        verbose=False,
    )

train_pool = Pool(Xtr, ytr, weight=wtr, cat_features=cat_cols)
model = make_model()
model.fit(train_pool)

p_test = model.predict_proba(Xte)[: , 1]
pr_auc = average_precision_score(yte, p_test, sample_weight=wte)
roc_auc = roc_auc_score(yte, p_test, sample_weight=wte)

# Baseline (a): constant base-rate predictor -> PR-AUC equals the weighted base rate
pr_auc_const = average_precision_score(yte, np.full(len(yte), base_rate_test),
sample_weight=wte)

# Baseline (b): single-feature cpv4 rate learned on train (weighted), applied to test
cpv4_tr = pd.DataFrame({'cpv4': Xtr['cpv_main__cpv4'], 'y': ytr, 'w': wtr})
rate = cpv4_tr.groupby('cpv4').apply(lambda g: np.average(g['y'], weights=g['w']),
include_groups=False)
train_base = np.average(ytr, weights=wtr)
p_cpv4 = Xte['cpv_main__cpv4'].map(rate).fillna(train_base).values
pr_auc_cpv4 = average_precision_score(yte, p_cpv4, sample_weight=wte)
roc_auc_cpv4 = roc_auc_score(yte, p_cpv4, sample_weight=wte)

print(f'CatBoost      PR-AUC: {pr_auc:.4f} | ROC-AUC: {roc_auc:.4f}')
print(f'constant base PR-AUC: {pr_auc_const:.4f} (= weighted test base rate
{base_rate_test:.4f})')
print(f'cpv4-only      PR-AUC: {pr_auc_cpv4:.4f} | ROC-AUC: {roc_auc_cpv4:.4f}')

CatBoost      PR-AUC: 0.3405 | ROC-AUC: 0.6703
constant base PR-AUC: 0.1667 (= weighted test base rate 0.1667)
cpv4-only      PR-AUC: 0.2539 | ROC-AUC: 0.6403

```

How good is it — train vs test, and the two ways to be wrong.

A prediction can fail in exactly two directions, and they have standard names:

- A **false positive** (a false alarm): the model flags a lot, but plenty of bidders show up. Cost to a contractor: you prepared a bid hoping to be alone, and walked into a price war.
- A **false negative** (a missed easy win): a lot ends with 0–1 bids, but the model never flagged it. Cost: an easy contract passed by unnoticed.

Precision and recall — defined in the Part 2 intro above — count exactly these two failure modes. At the 0.5 cut-off on test: precision 37 of 100 means **63 of every 100 flags are false positives**; recall 25 of 100 means **75 of every 100 single-bid lots are false negatives** that slip through unflagged.

The *train* row will look near-perfect. That is **memorization** (the textbook word is *overfitting*): the model has learnt its own training examples by heart, like a student who memorized last year's exam — expected for this model type, and meaningless. The **test** row is the only honest one, because those lots never influenced the model.

```
In [ ]: # Train vs test comparison (overfitting check).
# PR-AUC folds every precision/recall trade-off into one number; the @0.5 line shows
# one concrete operating point: of the lots the model flags, how many really were
# single-bid (precision), and how many of the real single-bid lots it caught
# (recall).
from sklearn.metrics import precision_score, recall_score

p_train = model.predict_proba(Xtr)[: , 1]
for name, yy, pp, ww in [('train', ytr, p_train, wtr), ('test ', yte, p_test, wte)]:
    pr = average_precision_score(yy, pp, sample_weight=ww)
    roc = roc_auc_score(yy, pp, sample_weight=ww)
    yhat = (pp >= 0.5).astype(int)
    prec = precision_score(yy, yhat, sample_weight=ww)
    rec = recall_score(yy, yhat, sample_weight=ww)
    base = np.average(yy, weights=ww)
    print(f'{name} | base rate {base:.3f} | PR-AUC {pr:.4f} | ROC-AUC {roc:.4f} | '
          f'@0.5 cut-off: precision {prec:.4f}, recall {rec:.4f}')
```

```
train | base rate 0.089 | PR-AUC 0.8936 | ROC-AUC 0.9925 | @0.5 cut-off: precision
0.6541, recall 0.9964
test  | base rate 0.167 | PR-AUC 0.3405 | ROC-AUC 0.6703 | @0.5 cut-off: precision
0.3712, recall 0.2494
```

What drives the predictions. The next cell prints three checks as plain sentences; the only decoding needed is the feature names: a double underscore marks an engineered fact — `cpv_main__cpv4` = the trade code cut to its first 4 digits (4523... = road works), `__nuts2 / __nuts3` = EU region levels ($\approx$ district / county), `span__X` = days between publication and date X. Plain names are facts straight from the notice (`duration_days` , `cv_required` , ...).

What to watch for in the output: **no single fact dominates** (many weak clues = honest signal, see trust check 3); **reality rises with the claim** (the ranking is trustworthy); but **the top-end claims run about double reality** — which is exactly why customers will see HIGH/MEDIUM/LOW tiers instead of raw percentages.

```

In [ ]: # Three checks, printed as plain sentences: what the model leans on, whether its
# claimed percentages are honest overall, and whether they are honest per sector.
imp = pd.Series(model.get_feature_importance(train_pool),
index=model.feature_names_.sort_values(ascending=False))
print('Which facts the model leans on (share of its decision-making; all 82 facts sum
to 100%):')
for f, v in imp.head(15).items():
    print(f' {v:4.1f}% {f}')

from sklearn.calibration import calibration_curve
frac_pos, mean_pred = calibration_curve(yte, p_test, n_bins=8, strategy='quantile')
print('
Reality check of the claimed risk (test lots in 8 groups, lowest to highest claim):')
for i, (mp, fp) in enumerate(zip(mean_pred, frac_pos), 1):
    print(f' group {i}: model claimed {mp*100:3.0f} in 100 would end single-bid ->
in reality {fp*100:3.0f} in 100 did')
print(f' => ordering is right: reality rises from {frac_pos[0]*100:.0f} in 100
(group 1) to {frac_pos[-1]*100:.0f} in 100 (group 8).')
print(f' => but the top claim is {mean_pred[-1]/frac_pos[-1]:.1f}x reality
({mean_pred[-1]*100:.0f} claimed vs {frac_pos[-1]*100:.0f} actual):')
print(' trust the order, not the number – hence tiers for customers, not raw
percentages.')

SECTOR = {'450': 'general construction', '451': 'site preparation', '452': 'civil
engineering',
'453': 'building installation', '454': 'finishing trades'}
print('
Same reality check per construction sub-sector (>= 20 test lots):')
grp = pd.DataFrame({'cpv3': Xte['cpv_main__cpv3'], 'y': yte, 'p': p_test, 'w': wte})
for cpv3, g in grp.groupby('cpv3'):
    if len(g) < 20:
        continue
    obs = np.average(g['y'], weights=g['w']) * 100
    prd = np.average(g['p'], weights=g['w']) * 100
    print(f' {cpv3} {SECTOR.get(cpv3, ""):22s}: model claims {prd:3.0f} in 100,
reality {obs:3.0f} in 100 ({len(g)} lots)')

```

Which facts the model leans on (share of its decision-making; all 82 facts sum to 100%):

- 7.1% cpv_main__cpv4
- 5.4% duration_days
- 4.6% cpv_main__cpv3
- 3.3% place_nuts3__nuts2
- 3.3% bid_validity_raw
- 3.1% cpv_additional__cpv4
- 3.1% place_nuts3__nuts3
- 2.8% selection_criteria_types
- 2.7% bid_validity_days
- 2.6% span__period_end
- 2.6% span__period_start
- 2.5% cv_required
- 2.4% buyer_activity
- 2.4% buyer_nuts__nuts2
- 2.4% cpv_additional__cpv3

Reality check of the claimed risk (test lots in 8 groups, lowest to highest claim):

group 1: model claimed 5 in 100 would end single-bid -> in reality 7 in 100 did
group 2: model claimed 9 in 100 would end single-bid -> in reality 8 in 100 did
group 3: model claimed 12 in 100 would end single-bid -> in reality 11 in 100 did
group 4: model claimed 17 in 100 would end single-bid -> in reality 16 in 100 did
group 5: model claimed 21 in 100 would end single-bid -> in reality 15 in 100 did
group 6: model claimed 28 in 100 would end single-bid -> in reality 17 in 100 did
group 7: model claimed 40 in 100 would end single-bid -> in reality 22 in 100 did
group 8: model claimed 72 in 100 would end single-bid -> in reality 32 in 100 did
=> ordering is right: reality rises from 7 in 100 (group 1) to 32 in 100 (group 8).
=> but the top claim is 2.2x reality (72 claimed vs 32 actual):
trust the order, not the number – hence tiers for customers, not raw percentages.

Same reality check per construction sub-sector (>= 20 test lots):

450 general construction	: model claims 16 in 100, reality 13 in 100	(98 lots)
451 site preparation	: model claims 7 in 100, reality 6 in 100	(80 lots)
452 civil engineering	: model claims 35 in 100, reality 19 in 100	(397 lots)
453 building installation	: model claims 24 in 100, reality 24 in 100	(218 lots)
454 finishing trades	: model claims 17 in 100, reality 9 in 100	(171 lots)

Finding 1 — the score is an honest ranking, not an honest percentage

The model outputs one number between 0 and 1 for every tender — the **score**, intended to mean "the chance this lot ends with 0–1 bids". The reality check above shows this score has two different qualities at the same time:

- **As a comparison, it is truthful.** The higher the score, the more likely the tender really is a lonely one. Walking up the eight groups the real lonely-rate climbs 7 → 8 → 11 → 16 → 15 → 17 → 22 → 32 per 100 and never breaks the order.
- **As a number, it exaggerates at the top.** Where the model claims "72 in 100", reality is 32 in 100 — about twice too high. Below claims of ~17 in 100 it is honest.

Like a thermometer that is accurate at room temperature but reads double inside an oven: never quote its display, but "pot A reads hotter than pot B" is still true. **Trust the order, not the number.**

Why it exaggerates — by design, not by accident. Lonely lots are rare (about 1 in 10), so training deliberately punished *missing* one much harder than raising a false alarm — without that, the model would learn that always answering "competitive" is 90% correct and would spot nothing. The price of forcing it to look is a good spotter with an exaggerated voice.

Finding 2 — the sorting works inside a single trade (a sorter, not a teller)

Finding 1 condenses into one sentence: **the model is much better at *sorting* tenders by competitiveness than at *telling* the probability.** So all product value comes from the ordering — show a customer the top of *their* list, never the number.

But a real customer cannot change their business: a painter competes with painters, whatever we predict. So the decisive question is whether the sorting still works **inside one trade** — or whether the model's only trick was knowing that some trades are lonelier than others. The next cell measures exactly that, then compares two opposite worlds:

- a **gold-mine business** — building installation (electrics, plumbing, HVAC), where every 4th tender ends lonely, and
- a **cut-throat business** — finishing trades (painting, drywall, joinery), where only every 11th does.

If the model can pick the lonely tenders over the competitive ones *in both worlds*, the shortlist product works for both kinds of customer — it cannot change the world a contractor competes in, but inside any world it can point at the lonelier tenders.

```

In [ ]: # Sorting power INSIDE one trade: within each sub-sector, rank only that sector's
# own test lots and check whether our top 20% beats the sector's own base rate.
# (AUC-inside: 0.5 = coin flip, higher = the ordering works within the trade.)
from sklearn.metrics import roc_auc_score as _roc_auc

rows = pd.DataFrame({'cpv3': Xte['cpv_main__cpv3'].values, 'y': yte, 'p': p_test,
'w': wte})
stats = {}
print('Within-trade sorting power (test lots, sectors with >= 50 lots):')
for cpv3, g in rows.groupby('cpv3'):
    if len(g) < 50 or g['y'].nunique() < 2:
        continue
    base = np.average(g['y'], weights=g['w'])
    auc = _roc_auc(g['y'], g['p'], sample_weight=g['w'])
    top = g.nlargest(max(1, int(len(g) * 0.2)), 'p')
    hit = np.average(top['y'], weights=top['w'])
    stats[cpv3] = (base, auc, hit)
    note = 'no reliable skill' if auc < 0.55 else f'lift {hit/base:.1f}x'
    print(f' {cpv3} {SECTOR.get(cpv3, ""):22s}: {len(g):3d} lots | base
{base*100:2.0f} in 100 | '
        f'AUC-inside {auc:.2f} | top-20% picks hit {hit*100:2.0f} in 100 ({note})')

for label, cpv3, biz in [('GOLD MINE ', '453', 'an electrical installation
contractor'),
                        ('CUT-THROAT', '454', 'a painting / drywall contractor')]:
    base, auc, hit = stats[cpv3]
    print(f'
{label} - {SECTOR[cpv3]} ({biz}):')
    print(f' picking at random in this trade: {base*100:2.0f} in 100 tenders end
lonely')
    print(f' picking from our top 20% : {hit*100:2.0f} in 100 tenders end
lonely -> {hit/base:.1f}x the odds')

print('
=> In BOTH worlds the model picks lonely tenders over competitive ones (~1.8x the
odds).')
print(' It cannot change the world a contractor competes in; inside any world it
points at the lonelier tenders.')
print(' Honest exceptions: general construction and site preparation – ranking
there is currently')
print(' a coin flip; no product for those trades yet.')

```

Within-trade sorting power (test lots, sectors with ≥ 50 lots):

450 general construction : 98 lots | base 13 in 100 | AUC-inside 0.51 | top-20% picks hit 21 in 100 (no reliable skill)

451 site preparation : 80 lots | base 6 in 100 | AUC-inside 0.46 | top-20% picks hit 0 in 100 (no reliable skill)

452 civil engineering : 397 lots | base 19 in 100 | AUC-inside 0.67 | top-20% picks hit 37 in 100 (lift 1.9x)

453 building installation : 218 lots | base 24 in 100 | AUC-inside 0.68 | top-20% picks hit 43 in 100 (lift 1.8x)

454 finishing trades : 171 lots | base 9 in 100 | AUC-inside 0.60 | top-20% picks hit 16 in 100 (lift 1.8x)

GOLD MINE – building installation (an electrical installation contractor):

picking at random in this trade: 24 in 100 tenders end lonely

picking from our top 20% : 43 in 100 tenders end lonely -> 1.8x the odds

CUT-THROAT – finishing trades (a painting / drywall contractor):

picking at random in this trade: 9 in 100 tenders end lonely

picking from our top 20% : 16 in 100 tenders end lonely -> 1.8x the odds

=> In BOTH worlds the model picks lonely tenders over competitive ones (~1.8x the odds).

It cannot change the world a contractor competes in; inside any world it points at the lonelier tenders.

Honest exceptions: general construction and site preparation – ranking there is currently

a coin flip; no product for those trades yet.

Part 3 — Trust checks (the four tripwires)

Models like this usually cheat by accident — some field quietly contains the answer, and the numbers look great until real money is on the line. Each check below is designed to catch one way of cheating.

Check 1 — scrambled answers. We deliberately shuffle the outcomes and retrain. Now there is nothing real to learn, so the score **must** collapse to chance level. If it stayed high, the pipeline itself would be feeding answers to the model.

```
In [ ]: # Tripwire 1 – shuffled-label run: permute labels at LOT level (revisions of a lot
keep
# a common, but randomly reassigned, label), retrain, evaluate on the untouched test
set.
# With nothing to learn the score MUST collapse to the base rate.
rng = np.random.default_rng(42)
train_lot_ids = data.loc[is_train.values, key].apply(tuple, axis=1)
uniq = train_lot_ids.drop_duplicates().tolist()
lot2label = dict(zip(map(tuple, data[key].apply(tuple, axis=1)), data['label']))
orig = np.array([lot2label[l] for l in uniq])
shuffled = rng.permutation(orig)
shuf_map = dict(zip(uniq, shuffled))
ytr_shuf = train_lot_ids.map(shuf_map).values

model_shuf = make_model()
model_shuf.fit(Pool(Xtr, ytr_shuf, weight=wtr, cat_features=cat_cols))
p_shuf = model_shuf.predict_proba(Xte)[: , 1]
pr_shuf = average_precision_score(yte, p_shuf, sample_weight=wte)
roc_shuf = roc_auc_score(yte, p_shuf, sample_weight=wte)
print(f'shuffled-label PR-AUC: {pr_shuf:.4f} (base rate {base_rate_test:.4f}) | ROC-
AUC: {roc_shuf:.4f} (chance 0.5)')
assert pr_shuf < base_rate_test * 1.5, 'TRIPWIRE: shuffled-label score did not
collapse – pipeline leaks answers'
print('tripwire 1 PASSED: score collapsed to base rate')
```

```
shuffled-label PR-AUC: 0.1883 (base rate 0.1667) | ROC-AUC: 0.5414 (chance 0.5)
tripwire 1 PASSED: score collapsed to base rate
```

Check 2 — too good to be true. Published research on predicting competition at publication time tops out around ROC-AUC ≈ 0.7 (the "pairwise bet" score defined at the start of Part 2). A score far above that doesn't mean genius — it means a leak, until proven otherwise.

Check 3 — one-field wonder. Train a tiny model on each field *alone*. If any single field nearly matches the full model, that field probably encodes the outcome through a back door. Real signal here should be many weak clues, not one magic column.


```
In [ ]: # Tripwire 2 – too-good alarm: literature tops out ~ROC-AUC 0.7 for call-time
        # prediction; a near-perfect score means a leak until a specific feature is
        # exonerated.
        print(f'full-model ROC-AUC: {roc_auc:.4f}')
        assert roc_auc < 0.85, 'TRIPWIRE: score too good to be true – hunt the leaking
        feature'
        print('tripwire 2 PASSED: within plausible range\n')

        # Tripwire 3 – single-feature audit: train on each feature alone; one feature scoring
        # near the full model is suspicious (real signal here is many weak features).
        # Constant features (homogeneous corpus, see Known caveats) cannot be trained on –
        # skipped.
        def single_feature_score(col):
            is_cat = col in cat_cols
            m = CatBoostClassifier(
                cat_features=[col] if is_cat else [],
                one_hot_max_size=ONE_HOT_MAX_SIZE,
                auto_class_weights='Balanced',
                iterations=200, random_seed=42, verbose=False,
            )
            m.fit(Pool(Xtr[[col]], ytr, weight=wtr, cat_features=[col] if is_cat else []))
            p = m.predict_proba(Xte[[col]][:, 1])
            return average_precision_score(yte, p, sample_weight=wte)

        constant = [c for c in FEATURES if Xtr[c].nunique(dropna=False) <= 1]
        print('skipped as constant in train:', constant)
        audit = pd.Series({c: single_feature_score(c) for c in FEATURES if c not in
        constant}).sort_values(ascending=False)
        print('\ntop 12 single-feature PR-AUCs (full model: %.4f, base: %.4f):' % (pr_auc,
        base_rate_test))
        print(audit.head(12).round(4).to_string())
        suspicious = audit[audit > 0.9 * pr_auc]
        print('\nfeatures within 90% of full model:', list(suspicious.index) if
        len(suspicious) else 'none')
        print('tripwire 3', 'REVIEW NEEDED' if len(suspicious) else 'PASSED: no single
        feature rivals the full model')
```

full-model ROC-AUC: 0.6703

tripwire 2 PASSED: within plausible range

skipped as constant in train: ['notice_kind', 'contract_type', 'cpv_main__cpv2',
'procedure_languages', 'buyer_country', 'n_doc_references']

top 12 single-feature PR-AUCs (full model: 0.3405, base: 0.1667):

buyer_nuts__nuts3	0.2575
cpv_main__cpv4	0.2539
buyer_nuts__nuts1	0.2392
buyer_nuts__nuts2	0.2372
buyer_legal_type	0.2371
buyer_activity	0.2319
selection_criteria_types	0.2304
exclusion_grounds	0.2279
duration_days	0.2233
bid_validity_raw	0.2216
n_criteria_financial	0.2163
notice_subtype	0.2153

features within 90% of full model: none

tripwire 3 PASSED: no single feature rivals the full model

Check 4 — dress rehearsal. Score today's still-open tenders, where no outcome exists yet. If any model input could not be computed for them, that input secretly depended on the future — and the whole model would be unusable in practice. Passing means the model works on day one, which is the whole product.

```
In [ ]: # Tripwire 4 – production dry-run: score still-open tenders (no award exists yet).
# Any feature that cannot be computed for them depended on the future.
awarded_keys = set(map(tuple, aw_label[key].values))
open_mask = ~tenders[key].apply(tuple, axis=1).isin(awarded_keys)
open_tenders = tenders[open_mask].copy()
print(f'still-open tender rows: {len(open_tenders)}
      ({open_tenders.groupby(key).ngroups} lots)')

X_open, cats_o, nums_o, _ = build_features(open_tenders)
assert cats_o + nums_o == FEATURES, 'feature set differs for open tenders – some
feature depends on the future'
assert list(X_open.columns) == list(X.columns)
p_open = model.predict_proba(X_open)[: , 1]
print('all features computable for open tenders; scores produced for every row')
print('score distribution on open tenders:')
print(pd.Series(p_open).describe().round(3).to_string())
print(f'\nshare of open lots flagged above 0.5: {(p_open > 0.5).mean():.3f}')
print('tripwire 4 PASSED')
```

```
still-open tender rows: 13797 (11469 lots)
all features computable for open tenders; scores produced for every row
score distribution on open tenders:
count    13797.000
mean         0.242
std          0.186
min          0.002
25%         0.107
50%         0.190
75%         0.322
max          0.967

share of open lots flagged above 0.5: 0.102
tripwire 4 PASSED
```

What to do next

The product: a weekly shortlist, not a prediction machine. Each customer names their trades and region; every week we show the ~20 highest-ranked open tenders inside that box — never raw scores (the number lies exactly at the top; the order does not). On today's data such a shortlist roughly **doubles the odds** of standing alone at a tender — in gold-mine and cut-throat trades alike — while cutting the customer's reading from ~600 notices a week to 20.

Who to talk to first: a building-services installation contractor (electrics, plumbing, HVAC) — the trade with the most lonely tenders (every 4th) *and* the strongest sorting (our top picks: 43 in 100). The one-sentence pitch: *"In your trade, 1 tender in 4 ends with at most one bidder. Among our picks: almost every second."* Second door: civil engineering (Tiefbau). Not yet: general construction and site preparation — the sorting has no skill there today, and we say so.

Already running — this is not a one-off study. A weekly automated cycle downloads every new German construction tender (fresh within a day), retrains, writes every prediction into an append-only ledger *before* the outcome is knowable, and grades itself as awards arrive. Within weeks this produces the strongest sales asset possible: a **published, verifiable track record** — "our picks ended with at most one bidder about twice as often as chance."

Next build steps, in order:

1. **Per-customer shortlists** — a trade × region filter on the existing ranking; small work, immediate product.
2. **A calibration layer** — a correction table learned from the reality checks above ("when the model says 72, the honest number is 32"), enabling **HIGH / MEDIUM / LOW tiers** whose plain meaning anyone can verify.
3. **A pilot with one installation contractor**, graded week by week by the ledger — the pilot's own track record becomes the sales material for the next customer.